

one can use the PEER role to restrict endorsement to only peers. For example:

```
peer chaincode instantiate -C <channelid> -n mycc -P
"AND('Org1.peer', 'Org2.peer')"
```

A new organisation added to the channel after instantiation can query a chaincode -- provided the query has the appropriate authorisation as defined by channel policies and any application level checks enforced by the chaincode -- but will not be able to execute or endorse the chaincode. The endorsement policy needs to be modified to allow transactions to be committed with endorsements from the new organisation.

Endorsement policy syntax

Policies are expressed in terms of principals, which are identities matched to a role. Principals are described as 'MSP.ROLE', where MSP represents the required MSP ID and ROLE represents one of the four accepted roles: member, admin, client, and peer.

Examples of valid principals are given below:

```
'Org0.admin': any administrator of the Org0 MSP
'Org1.member': any member of the Org1 MSP
'Org1.client': any client of the Org1 MSP
'Org1.peer': any peer of the Org1 MSP
```

The syntax of the language is:

```
EXPR(E[, E...])
```

In the above case, *EXPR* is either *AND*, *OR*, or *OutOf*, and *E* is either a principal (with the syntax described above) or another nested call to *EXPR*. For example, *AND('Org1.member', 'Org2.member', 'Org3.member')* requests one signature from each of the three principals; and *OR('Org1.member', 'Org2.member')* requests one signature from either one of the two principals.

Key-level endorsement policies

Setting regular chaincode-level endorsement policies is tied to the life cycle of the corresponding chaincode. They can only be set or modified when instantiating or upgrading the corresponding chaincode on a channel. In contrast, key-level endorsement policies can be set and modified in a more granular fashion from within a chaincode. The modification is part of the read-write set of a regular transaction. The shim API provides the following functions to set and retrieve an endorsement policy for/from a regular key. For example:

```
SetStateValidationParameter(key string, ep []byte) error
GetStateValidationParameter(key string) ([]byte, error)
```

In the above code, 'ep' stands for the endorsement policy, which can be expressed either by using the same syntax described in chaincode-level policies or by using the convenience function described above. Either method will generate a binary version of the endorsement policy that can be consumed by the basic shim API. For keys that are part of private data in a collection, the following functions apply:

```
SetPrivateDataValidationParameter(collection, key string, ep
[]byte) error
GetPrivateDataValidationParameter(collection, key string) ([]
byte, error)
```

To help set endorsement policies and marshal them into validation parameter byte arrays, the Go shim provides an extension with convenience functions that allow the chaincode developer to deal with endorsement policies in terms of the MSP identifiers of organisations. For example, to set an endorsement policy for a key where two specific organisations are required to endorse the key change, pass both org MSPIDs to *AddOrgs()*, and then call *Policy()* to construct the endorsement policy byte array that can be passed to *SetStateValidationParameter()*.

Validation

At commit time, setting a value of a key is no different from setting the endorsement policy of a key — both update the state of the key and are validated based on the same rules. If a key is modified and no key-level endorsement policy is present, the chaincode-level endorsement policy applies, by default. This is also true when a key-level endorsement policy is set for a key for the first time — the new key-level endorsement policy must first be endorsed according to the pre-existing chaincode-level endorsement policy.

If a key is modified and a key-level endorsement policy is present, the key-level endorsement policy overrides the chaincode-level endorsement policy. This means that the key-level endorsement policy can be either less restrictive or more restrictive than the chaincode-level endorsement policy. Because the chaincode-level endorsement policy must be satisfied in order to set a key-level endorsement policy for the first time, no trust assumptions have been violated.

If a key's endorsement policy is removed, the chaincode-level endorsement policy becomes the default again. If a transaction modifies multiple keys with different associated key-level endorsement policies, all of these policies need to be satisfied for the transaction to be valid. **END** 

By: Swapnil Kulkarni

The author is an open source enthusiast with experience in blockchain, cloud native solutions, containers and enterprise software product architectures. He has diverse experience with different private and hybrid cloud architectures. He is a technology hobbyist and writes at cloudnativetech.wordpress.com.